REQUIREMENTS ANALYSIS DOCUMENT
for
AUTOMATED GHERKIN AND BDD TEST SCENARIO GENERATOR

Issue 1.0

October 30, 2026

Prepared by

Murat Taş

Berat Yılmaz

Halil Buğra Şen

TABLE OF CONTENTS

1 Introduction....3
1.1 Purpose of the system....3
1.2 Scope of the system....3
1.3 Objectives and success criteria of the project....3
1.4 Definitions, acronyms, and abbreviations....4
1.5 References....4
1.6 Overview....5
2 Current system....6
3 Proposed system....7
3.1 Overview....7
3.2 Functional requirements....7
3.3 Nonfunctional requirements....10
3.3.1 Usability....10
3.3.2 Reliability....10
3.3.3 Performance....10
3.3.4 Supportability....11
3.3.5 Implementation....11
3.3.6 Interface....11
3.3.7 Packaging....11
3.3.8 Legal....11
3.4 System models....12
3.4.1 Scenarios....12
3.4.2 Use case model....14
3.4.3 Object model....17
3.4.4 Dynamic model....20
3.4.5 User interfaces and Mock-ups....23
4 Glossary of Terms....26
5 Priority....27
6 Traceability....28

1 Introduction

1.1 Purpose of the system

The **Automated Gherkin and BDD Test Scenario Generator** (hereafter "the system") is a developer-oriented software tool that transforms structured software requirements into Behavior-Driven Development (BDD) test scenarios. The system accepts requirements written in the Easy Approach to Requirements Syntax (EARS) and in the standard user story template, validates them, builds a format-independent internal representation of each requirement and generates syntactically correct Gherkin feature files that follow the Given-When-Then structure.

Writing acceptance scenarios by hand is slow, repetitive and inconsistent between team members, and the link between a requirement and the scenarios that test it is easily lost. The system is developed to automate this transformation so that development and test teams obtain a consistent first set of executable-style scenarios directly from their requirements, including scenarios for alternative and exceptional flows that are often forgotten during manual writing. The project is also the vehicle through which the team applies object-oriented analysis and design (encapsulation, abstraction, inheritance, polymorphism and composition) and the Builder, Adapter and Chain of Responsibility design patterns to a realistic software-engineering automation problem.

1.2 Scope of the system

The scope of the system covers the complete transformation pipeline from requirement text to Gherkin text. The following capabilities are **in scope**:

- Entry of single requirements and import of requirement collections from text files.
- Support for two requirement styles: EARS (all six patterns) and user stories.
- Validation of requirements through an ordered chain of processing stages with meaningful error feedback.
- Construction of a unified requirement model and a requirement syntax tree.
- Generation of normal, alternative and exceptional scenarios and their formatting as Gherkin text.
- Viewing, copying and exporting scenarios as .feature files, and reviewing the trace between requirement elements and scenario steps.
- Optional storage of a generation history.

The following items are **out of scope** for this version but are anticipated by the design: execution of the generated scenarios (step definitions and test runners), free natural-language requirements that follow neither EARS nor the user story template, AI-assisted requirement interpretation, BDD dialects other than English Gherkin, other test-generation targets (for example JUnit test skeletons) and multi-user collaboration.

1.3 Objectives and success criteria of the project

The objectives of the project and the criteria by which their achievement will be judged are listed in the table below.

Table 1. Objectives and success criteria

ID	Objective	Success criterion
O-1	Convert EARS requirements and user stories into Gherkin scenarios automatically.	All 6 EARS patterns and the user story template are accepted; for a reference set of 40 valid requirements, 100% produce at least one scenario.
O-2	Produce syntactically correct and meaningful scenarios.	100% of generated .feature files are parsed without error by the official Cucumber Gherkin parser; in a team review at least 80% of scenarios are judged usable without rewording.
O-3	Give useful feedback on invalid requirements.	For a reference set of 20 defective requirements, every defect is reported with stage, problematic input, reason and suggested correction.
O-4	Cover alternative and exceptional flows.	For every unwanted-behavior EARS requirement and every requirement with a numeric limit, at least one non-normal scenario is generated.
O-5	Keep the design extensible.	A third requirement format can be added by writing one parser stage extension and one FormatAdapter, with zero changes to the scenario-construction classes (verified in Deliverable #3).
O-6	Demonstrate sound OO analysis and design.	Builder, Adapter and Chain of Responsibility each solve an identified design problem and are traceable to requirements in this document.

1.4 Definitions, acronyms, and abbreviations

The acronyms and abbreviations used in this document are listed below; domain terms are defined in the Glossary of Terms (Section 4).

Table 2. Acronyms and abbreviations

Term	Meaning
AST	Abstract Syntax Tree; called the requirement syntax tree in this document
BDD	Behavior-Driven Development
CoR	Chain of Responsibility design pattern
EARS	Easy Approach to Requirements Syntax
FR / NFR	Functional Requirement / Nonfunctional Requirement
GUI	Graphical User Interface
MoSCoW	Must have, Should have, Could have, Won't have (prioritization scheme)
RAD	Requirements Analysis Document
UC	Use Case
UML	Unified Modeling Language
UTF-8	Unicode Transformation Format, 8-bit

1.5 References

- [1] Automated Gherkin and BDD Test Scenario Generator, Project Problem Statement, CSE443 Object-Oriented Analysis and Design, 2026.
- [2] CSE443 Project Deliverable #1: Requirements Analysis Document, assignment description and RAD template, 2026.
- [3] B. Bruegge and A. H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java, 3rd ed., Prentice Hall, 2010.
- [4] A. Mavin, P. Wilkinson, A. Harwood and M. Novak, "Easy Approach to Requirements Syntax (EARS)", Proc. 17th IEEE International Requirements Engineering Conference (RE'09), 2009, pp. 317-322.
- [5] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
- [6] Cucumber, Gherkin Reference, https://cucumber.io/docs/gherkin/reference/.

- [7] M. Cohn, User Stories Applied: For Agile Software Development, Addison-Wesley, 2004.
- [8] Object Management Group, Unified Modeling Language Specification, Version 2.5.1, 2017.

1.6 Overview

Section 2 describes how BDD scenarios are produced today and the problems of the current practice. Section 3 presents the proposed system: a functional overview, the functional requirements grouped by module and written in EARS form, the nonfunctional requirements and the system models (scenarios, use case model, object model, dynamic model and user interface mock-ups). Section 4 is the glossary, Section 5 assigns priorities to the requirements and Section 6 traces the requirements to the problem statement, to the use cases and to the analysis classes.

2 Current system

There is no existing automated system in the team's context; the tasks supported by the new system are currently performed manually. In a typical BDD workflow a business analyst or product owner writes requirements as user stories (in an issue tracker such as Jira) or as "shall" statements in a specification document. A developer or test engineer then reads each requirement and writes Gherkin scenarios by hand in a .feature file, usually during a refinement meeting or just before implementation. The scenarios are finally bound to step definitions and executed by a BDD runner such as Cucumber or SpecFlow.

Existing tools support only the end of this chain. BDD runners execute scenarios but do not create them; IDE plug-ins offer syntax highlighting and keyword completion for Gherkin; requirement management tools store requirements but do not derive tests from them; general-purpose AI chat assistants can draft scenarios but produce different output for the same input and give no structured explanation of why a requirement is defective. The current practice therefore has the following problems:

- **Effort:** writing and maintaining scenarios for every requirement is time-consuming and repetitive.
- **Inconsistency:** different authors phrase steps differently, mix Given/When/Then responsibilities and use inconsistent indentation and naming.
- **Missing coverage:** authors concentrate on the successful path; invalid-input, missing-information, unauthorized and boundary scenarios are often omitted.
- **Late defect detection:** incomplete or ambiguous requirements (for example a missing actor or a missing system response) are discovered only when someone tries to write a scenario.
- **Lost traceability:** after editing, it is no longer clear which requirement and which part of it a given step tests.
- **Non-determinism of AI drafts:** generated text is not reproducible and cannot be reviewed against explicit transformation rules.

3 Proposed system

This section documents the requirements elicitation and the analysis model of the new system.

3.1 Overview

The proposed system is a desktop application organized as a layered transformation pipeline: **User Interface → Requirement Processing Pipeline → Unified Requirement Model → Scenario Construction → Gherkin Formatter**. The User enters or imports requirements; each requirement travels through an ordered chain of processing stages (Chain of Responsibility), each of which either processes it and passes it on or stops the chain with a structured error. During the chain, a format-specific FormatAdapter (Adapter) converts the EARS requirement or user story into a common RequirementModel, from which a requirement syntax tree is built. The scenario construction stage identifies the normal and alternative flows and uses a ScenarioBuilder (Builder) to assemble each GherkinScenario step by step, refusing to complete a scenario without the mandatory steps. Finally the GherkinFormatter turns the features into Gherkin text that the User can view, trace, copy and export.

The core transformation rules that the analysis model must support are summarized below; they define how the logical elements of a requirement, rather than its wording, are mapped to Gherkin steps.

Table 3. Mapping of requirement elements to Gherkin steps

Input form	Template	Given	When	Then
EARS ubiquitous	The <system> shall <response>.	the system is in normal operation	the operation concerned by the response occurs	the system should <response>
EARS event-driven	When <trigger>, the <system> shall <response>.	preconditions / actor context	<trigger>	the system should <response>
EARS state-driven	While <state>, the <system> shall <response>.	<state>	the relevant action occurs	the system should <response>
EARS unwanted behavior	If <condition>, then the <system> shall <response>.	actor context	<condition> occurs	the system should <response> (plus complementary scenario)
EARS optional feature	Where <feature>, the <system> shall <response>.	<feature> is included / enabled	the relevant action occurs	the system should <response>
EARS complex	Combination of While / When / Where / If clauses	all states and features	trigger or unwanted condition	response
User story	As a <role>, I want <capability>, so that <benefit>.	the user is a <role>	the user <capability>	<benefit> is achieved

3.2 Functional requirements

The functional requirements are grouped by module. Every requirement has a unique identifier and is written with the EARS sentence structure (ubiquitous: "The system shall…", event-driven: "When…, the system shall…", unwanted behavior: "If…, then the system shall…", optional feature: "Where…, the system shall…"). The term **User** denotes any human actor (Developer, Test Engineer or Business Analyst). Class names written in CamelCase refer to the object model in Section 3.4.3.

3.2.1 Requirement Input

ID	Requirement
FR-1	The system shall allow the User to enter a single requirement as text in the Requirement Editor.
FR-2	The system shall allow the User to import a requirement collection from a UTF-8 plain-text file in which requirements are separated by blank lines.
FR-3	The system shall accept EARS requirements written in the ubiquitous, event-driven, state-driven, unwanted-behavior, optional-feature and complex patterns.
FR-4	The system shall accept user stories written in the form "As a <role>, I want <capability>, so that <benefit>".
FR-5	The system shall allow the User to select the requirement format as Auto-detect, EARS or User Story before submission.
FR-6	When a requirement is submitted with the Auto-detect format, the system shall determine the requirement format from the keywords of the requirement.
FR-7	The system shall assign a unique requirement identifier to each submitted requirement.
FR-8	If a submitted requirement is empty, then the system shall reject the requirement and ask the User to enter requirement text.

3.2.2 Requirement Parsing and Representation

ID	Requirement
FR-9	When a requirement passes syntax validation, the system shall split the requirement into tokens.
FR-10	When an EARS requirement is tokenized, the system shall identify the EARS pattern of the requirement.
FR-11	When a user story is tokenized, the system shall identify the role, the capability and the benefit of the user story.
FR-12	The system shall extract the actor, action, preconditions, trigger, expected behavior, outcome and constraints that are present in a requirement.
FR-13	The system shall convert each format-specific requirement into a RequirementModel that does not depend on the original requirement format.
FR-14	When a RequirementModel is created, the system shall construct a syntax tree whose root is a RequirementNode.
FR-15	The system shall represent each extracted element as a typed syntax node (ActorNode, ActionNode, ConditionNode, TriggerNode, ExpectedBehaviorNode or OutcomeNode) that records the text span from which it was extracted.

3.2.3 Requirement Validation and Processing Pipeline

ID	Requirement
FR-16	The system shall process each requirement through the ordered processing stages Syntax Validation, Tokenization, Structural Validation, Syntax Tree Construction and Scenario Construction.
FR-17	When a processing stage completes successfully, the system shall pass the requirement to the next processing stage.
FR-18	If a processing stage detects an error, then the system shall stop processing the requirement and report the error.
FR-19	If a requirement does not match the syntax of its requirement format, then the system shall report a syntax error.
FR-20	If a requirement has no identifiable actor, then the system shall report a missing-actor error.
FR-21	If a requirement has no identifiable action, then the system shall report a missing-action error.
FR-22	If a requirement does not state an expected system behavior, then the system shall report an incomplete-expected-behavior error.
FR-23	If a requirement contains contradictory elements, then the system shall report a consistency warning that names the conflicting elements.
FR-24	The system shall allow the User to validate a requirement without generating scenarios.

3.2.4 Scenario Generation

ID	Requirement
FR-25	When a requirement passes structural validation, the system shall generate at least one GherkinScenario for the normal flow of the requirement.
FR-26	The system shall derive the Given steps of a scenario from the preconditions, states and actor of the requirement.
FR-27	The system shall derive the When steps of a scenario from the trigger and action of the requirement.
FR-28	The system shall derive the Then steps of a scenario from the expected behavior and outcome of the requirement.
FR-29	Where more than one requirement element maps to the same step type, the system shall add each additional element as an And step.
FR-30	When an EARS requirement follows the unwanted-behavior pattern, the system shall generate one scenario in which the unwanted condition occurs and one scenario in which it does not occur.
FR-31	Where an alternative-flow type (invalid input, missing information, unauthorized operation, boundary condition or exceptional behavior) is enabled, the system shall generate an additional scenario of that type for each related condition identified in the requirement.
FR-32	When a requirement contains a numeric limit, the system shall generate boundary scenarios for the value at the limit and the value immediately below the limit.
FR-33	The system shall compose step text from the actor, action and condition nodes of the syntax tree rather than copying the sentence of the requirement.
FR-34	If a scenario does not contain at least one When step and at least one Then step, then the system shall not complete the scenario and shall report a generation error.
FR-35	The system shall group the scenarios generated from one requirement into one GherkinFeature named after the capability of the requirement.

3.2.5 Output and Traceability

ID	Requirement
FR-36	The system shall format each GherkinFeature as Gherkin text that uses the keywords Feature, Scenario, Given, When, Then, And and But with two-space indentation per nesting level.
FR-37	The system shall display the formatted Gherkin text to the User.
FR-38	The system shall allow the User to copy the formatted Gherkin text to the clipboard.
FR-39	The system shall allow the User to export a GherkinFeature as a .feature file.
FR-40	When a requirement collection has been processed, the system shall allow the User to export all generated features as separate .feature files.
FR-41	The system shall record, for each GherkinStep, the syntax nodes from which the step was derived.
FR-42	When the User opens the Traceability view, the system shall display, for each step of the selected scenario, the requirement elements from which the step was derived.

3.2.6 Error Handling and User Feedback

ID	Requirement
FR-43	If processing of a requirement fails, then the system shall display an error report that states the problematic input, the processing stage, the reason for the failure and a suggested corrective action.
FR-44	The system shall classify each reported error as either an input error or a generation error.
FR-45	If an internal fault occurs during scenario construction, then the system shall report a generation error and keep the requirement text entered by the User.
FR-46	When a requirement in a requirement collection fails, the system shall continue processing the remaining requirements of the collection.
FR-47	When processing of a requirement collection completes, the system shall display a collection summary that lists the result of each requirement.

3.2.7 Generation History and Options

ID	Requirement
FR-48	Where history storage is enabled, the system shall store each processed requirement together with its format, syntax tree, generated features, scenarios and generation options as a GenerationRecord.
FR-49	Where history storage is enabled, the system shall allow the User to view, search and reload previous GenerationRecords.
FR-50	Where history storage is enabled, the system shall allow the User to delete a GenerationRecord.
FR-51	The system shall allow the User to enable or disable each alternative-flow type before generation.
FR-52	When a requirement collection is processed, the system shall apply the same generation options to every requirement of the collection.

3.3 Nonfunctional requirements

The nonfunctional requirements are numbered NFR-1 to NFR-32 and grouped according to the FURPS+ categories of the template.

3.3.1 Usability

ID	Requirement
NFR-1	The system shall allow the User to generate scenarios for an entered requirement with a single action (the Generate button or Ctrl+Enter).
NFR-2	The system shall express every error report in plain English without stack traces or internal identifiers.
NFR-3	The system shall highlight Gherkin keywords in the Scenario View.
NFR-4	The system shall provide one built-in example requirement for each supported EARS pattern and for the user story format.
NFR-5	A first-time User shall be able to generate and export a feature file within 5 minutes without consulting external documentation.

3.3.2 Reliability

ID	Requirement
NFR-6	If any input requirement is invalid, then the system shall continue running and shall remain usable.
NFR-7	The system shall produce identical Gherkin output for identical requirement text and generation options.
NFR-8	Every .feature file exported by the system shall be accepted by the official Cucumber Gherkin parser without errors.
NFR-9	If a processing error occurs, then the system shall preserve the requirement text entered by the User.

3.3.3 Performance

ID	Requirement
NFR-10	The system shall generate and display the scenarios of a single requirement within 1 second on a reference machine (4 CPU cores, 8 GB RAM).
NFR-11	The system shall process a collection of 100 requirements within 10 seconds on the reference machine.
NFR-12	While a requirement collection is being processed, the system shall display progress and keep the user interface responsive.

3.3.4 Supportability

ID	Requirement
NFR-13	The system shall allow a new requirement format to be added by adding a format-specific parsing extension and a FormatAdapter, without modifying the scenario construction classes.
NFR-14	The system shall allow a new processing stage or validation rule to be inserted into the processing chain without modifying the existing processing stages.
NFR-15	The system shall allow a new output format to be added by adding a formatter, without modifying GherkinFeature, GherkinScenario or GherkinStep.
NFR-16	The domain and processing classes shall be covered by automated unit tests with at least 80% statement coverage.
NFR-17	All public classes shall be documented with API comments, and the source code shall be maintained in the team's GitHub repository.

3.3.5 Implementation

ID	Requirement
NFR-18	The system shall be implemented in Java 17 following object-oriented principles (encapsulation, abstraction, inheritance, polymorphism and composition).
NFR-19	The system shall use the Chain of Responsibility pattern for the processing stages, the Adapter pattern for format conversion and the Builder pattern for the construction of GherkinScenario objects.
NFR-20	The system shall separate the user interface, processing pipeline, domain model, scenario construction and output formatting into distinct layers with dependencies directed only towards the domain model.
NFR-21	The core transformation shall not depend on any external NLP or AI service.
NFR-22	Where history storage is implemented, the system shall use an embedded relational database (SQLite) accessed only through a dedicated persistence layer.

3.3.6 Interface

ID	Requirement
NFR-23	The system shall read requirement collections from UTF-8 encoded plain-text (.txt) files.
NFR-24	The system shall write UTF-8 encoded .feature files that conform to the Gherkin syntax of the Cucumber reference [6].
NFR-25	The system shall provide a graphical desktop user interface built with JavaFX.
NFR-26	The system shall use the English Gherkin keywords Feature, Scenario, Given, When, Then, And and But.

3.3.7 Packaging

ID	Requirement
NFR-27	The system shall be delivered as a single executable JAR file that bundles all third-party libraries.
NFR-28	The GitHub repository shall contain the source code, a Maven build file, a README with build and run instructions, and sample requirement files for every supported format.
NFR-29	The system shall run on Windows, macOS and Linux with a Java 17 runtime.

3.3.8 Legal

ID	Requirement
NFR-30	The system shall use only third-party libraries released under licenses that permit academic redistribution (for example MIT or Apache 2.0), and the licenses shall be listed in the repository.
NFR-31	The system shall not transmit requirement text or generated scenarios to any external service.
NFR-32	All submitted work shall comply with the university's academic integrity rules; reused material shall be cited.

3.4 System models

This section contains the scenarios, the use case model, the object model, the dynamic model and the user interface mock-ups with the navigational paths between screens.

3.4.1 Scenarios

Scenarios are concrete, informal descriptions of single uses of the system from the point of view of one actor instance. They were used to elicit the use cases in Section 3.4.2.

Table 4. Scenario generateLoginScenarios

Scenario name	**generateLoginScenarios**
Participating actor instances	ayse: Developer
Flow of events	1. Ayşe is implementing the login feature of a web application. She opens the Requirement Editor, leaves the format on Auto-detect and types the user story "As a registered user, I want to log in with my e-mail and password, so that I can access my dashboard." 2. She presses Generate. The system detects the user story format, validates it, identifies the role (registered user), the capability (log in with e-mail and password) and the benefit (access the dashboard), and builds the syntax tree. 3. Because the Invalid input and Missing information alternative flows are enabled, the system generates three scenarios and shows the feature below in the Scenario View. 4. Ayşe copies the text to the clipboard and pastes it into the project's features folder.

Gherkin produced in scenario generateLoginScenarios:

```
Feature: Log in with e-mail and password
  As a registered user
  I want to log in with my e-mail and password
  So that I can access my dashboard

  Scenario: Registered user logs in successfully
    Given the user is a registered user
    When the user logs in with a valid e-mail and password
    Then the user should be able to access the dashboard

  Scenario: Login is rejected for invalid credentials
    Given the user is a registered user
    When the user logs in with an invalid e-mail or password
    Then the user should not be able to access the dashboard

  Scenario: Login is rejected when the password is missing
    Given the user is a registered user
    When the user logs in without entering a password
    Then the user should be informed that the password is required
```

Table 5. Scenario accountLockoutWithBoundaries

Scenario name	**accountLockoutWithBoundaries**
Participating actor instances	mehmet: Test Engineer
Flow of events	1. Mehmet must prepare acceptance tests for the security requirements. He enters the EARS requirement "If the user enters an invalid password three times, then the authentication service shall lock the user account for 15 minutes." 2. The system recognizes the unwanted-behavior pattern, extracts the condition (invalid password three times), the response (lock the user account) and the constraint (for 15 minutes). 3. The system generates a scenario in which the condition occurs, and, because the requirement contains the numeric limit 3 and the Boundary flow is enabled, a scenario with the value immediately below the limit (Figure 10). 4. Mehmet opens the Traceability tab, selects the first scenario and checks that the And step about 15 minutes comes from the constraint node (Figure 11). He then exports the feature as account_lockout.feature.

Table 6. Scenario incompleteRequirementFeedback

Scenario name	**incompleteRequirementFeedback**
Participating actor instances	zeynep: Business Analyst
Flow of events	1. Zeynep drafts a new requirement and wants to check it before handing it to the developers. She types "When the user clicks Save, the system shall." and presses Validate. 2. The syntax validation and tokenization stages succeed, but the structural validation stage finds no expected behavior after "shall" and stops the chain. 3. The system shows an Input Error report: stage Structural Validation (3 of 5), problematic input "the system shall .", reason "an event-driven requirement must state the system response", and the suggestion "When the user clicks Save, the system shall store the document." (Figure 12). 4. Zeynep presses Edit requirement, completes the sentence and validates again; this time all stages pass.

Table 7. Scenario importAndExportCollection

Scenario name	**importAndExportCollection**
Participating actor instances	ayse: Developer
Flow of events	1. At the start of a sprint Ayşe imports login-requirements.txt, which contains 12 requirements separated by blank lines. 2. The system processes the requirements one by one with the same generation options. Requirement REQ-0004 ("Login should be fast and nice.") matches no supported format and REQ-0006 has no expected behavior; both receive input errors, while the other 10 requirements produce 23 scenarios. 3. The Collection Summary lists each requirement with its result (Figure 13). Ayşe chooses Export all and selects a folder; the system writes one .feature file per generated feature.

Table 8. Scenario reloadFromHistory

Scenario name	**reloadFromHistory**
Participating actor instances	mehmet: Test Engineer
Flow of events	1. The day after scenario accountLockoutWithBoundaries, Mehmet needs the lockout feature again. He opens the History view, searches for "lockout" and selects yesterday's record. 2. The system reloads the requirement, its options and the generated feature into the editor and the Scenario View, exactly as they were generated.

3.4.2 Use case model

The use case model is shown in Figure 1. The primary actor **User** is specialized into Developer, Test Engineer and Business Analyst, who all have the same access to the system but different goals (implementation, acceptance testing and requirement quality). The secondary actor **File System** represents the operating-system file storage used for import and export. UC-4 always includes UC-3 because no scenario is generated from an unvalidated requirement; UC-8 extends UC-3 and UC-4 when a processing stage fails; UC-6 extends UC-5 when the User opens the traceability tab.

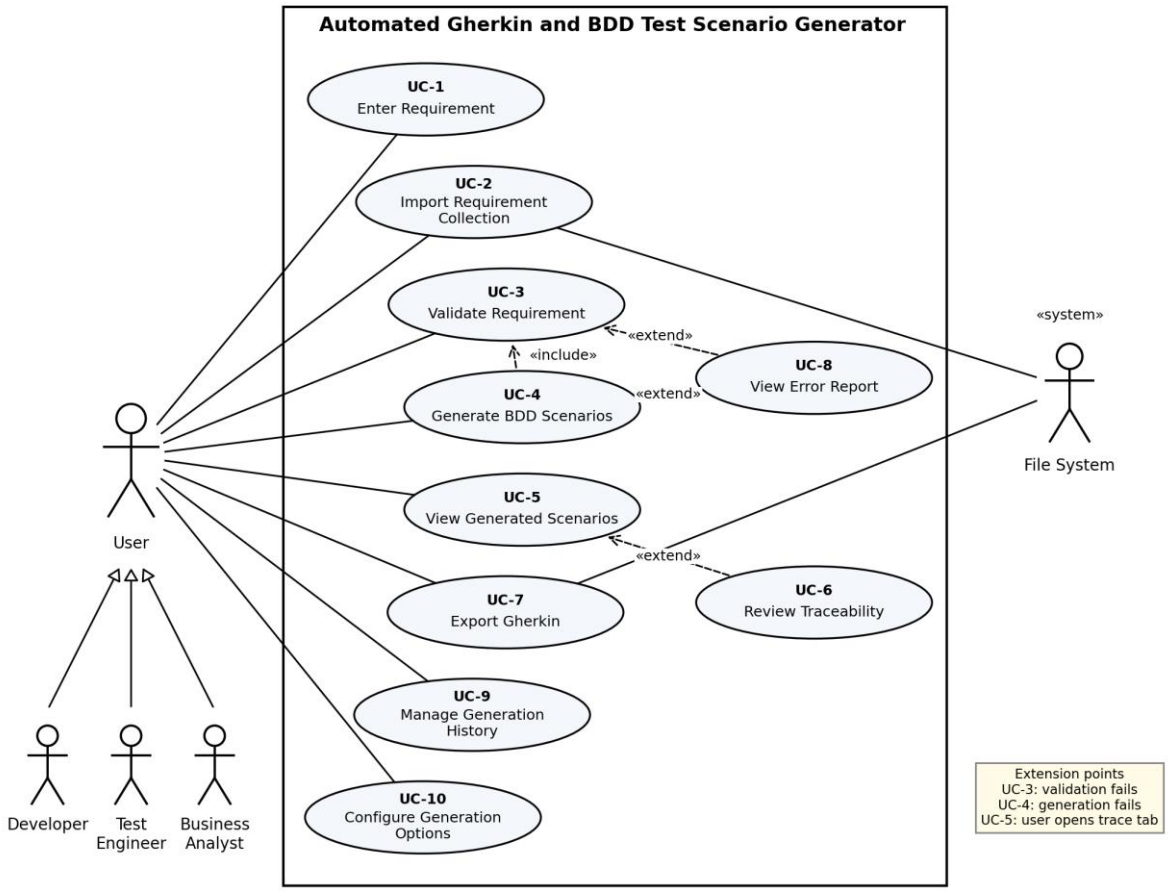


Figure 1. Use case diagram

The use cases are described below in the format of Bruegge and Dutoit [3].

Table 9. Use case UC-1 Enter Requirement

Use case	**UC-1 Enter Requirement**
Participating actors	User
Entry condition	The Requirement Editor is open.
Flow of events	1. The User selects a requirement format (Auto-detect, EARS or User Story). 2. The User types or pastes the requirement text. 3. The User submits the requirement by choosing Validate (UC-3) or Generate (UC- 4. The system assigns a requirement identifier and, for Auto-detect, determines the format.
Exit condition	The requirement is submitted with an identifier and a known format, or it is rejected because it is empty or its format is unsupported.
Quality requirements	Format detection completes within 100 ms.
Related requirements	FR-1, FR-3, FR-4, FR-5, FR-6, FR-7, FR-8

Table 10. Use case UC-2 Import Requirement Collection

Use case	**UC-2 Import Requirement Collection**
Participating actors	User, File System
Entry condition	The Requirement Editor is open.
Flow of events	1. The User chooses Import file. 2. The system shows the Import Dialog. 3. The User selects a UTF-8 text file. 4. The File System supplies the file content. 5. The system splits the file into requirements, assigns identifiers and processes each requirement with the current generation options (UC-4); failing requirements do not stop the collection. 6. The system displays the Collection Summary.
Exit condition	Every requirement of the file has a result (scenarios generated or error reported).
Quality requirements	100 requirements are processed within 10 s (NFR-11).
Related requirements	FR-2, FR-7, FR-46, FR-47, FR-52

Table 11. Use case UC-3 Validate Requirement

Use case	**UC-3 Validate Requirement**
Participating actors	User
Entry condition	A requirement has been submitted (UC-1).
Flow of events	1. The system checks the syntax of the requirement against its format. 2. The system splits the requirement into tokens and identifies the EARS pattern or the user-story parts. 3. The system checks that an actor, an action and an expected behavior are present and that the elements are consistent. 4. The system informs the User that the requirement is valid.
Exit condition	The requirement is marked Validated. Extension point: validation fails → UC-8.
Quality requirements	Validation does not modify the requirement text.
Related requirements	FR-9, FR-10, FR-11, FR-12, FR-16, FR-17, FR-18, FR-19, FR-20, FR-21, FR-22, FR-23, FR-24

Table 12. Use case UC-4 Generate BDD Scenarios

Use case	**UC-4 Generate BDD Scenarios**
Participating actors	User
Entry condition	A requirement has been submitted (UC-1) or imported (UC-2).
Flow of events	1. The system validates the requirement (includes UC-3). 2. The system adapts the format-specific requirement into a RequirementModel and builds the syntax tree. 3. The system identifies the normal flow and the enabled alternative flows. 4. For each flow the system builds a GherkinScenario with Given, When, Then and And/But steps. 5. The system groups the scenarios into a GherkinFeature, records the trace links and shows the result (UC-5).
Exit condition	A GherkinFeature with at least one complete scenario exists. Extension point: generation fails → UC-8.
Quality requirements	Identical input and options give identical output (NFR-7); result within 1 s (NFR-10).
Related requirements	FR-13, FR-14, FR-15, FR-16, FR-17, FR-25, FR-26, FR-27, FR-28, FR-29, FR-30, FR-31, FR-32, FR-33, FR-34, FR-35, FR-41

Table 13. Use case UC-5 View Generated Scenarios

Use case	**UC-5 View Generated Scenarios**
Participating actors	User
Entry condition	At least one GherkinFeature has been generated.
Flow of events	1. The system formats the feature as Gherkin text with keywords and indentation. 2. The system displays the text in the Gherkin tab of the Scenario View with keyword highlighting. 3. The User reads the scenarios and may continue with UC-6 or UC-7.
Exit condition	The formatted feature is visible. Extension point: the User opens the Traceability tab → UC-6.
Quality requirements	Keyword highlighting (NFR-3).
Related requirements	FR-36, FR-37

Table 14. Use case UC-6 Review Traceability

Use case	**UC-6 Review Traceability**
Participating actors	User
Entry condition	The User is viewing a generated feature (UC-5).
Flow of events	1. The User opens the Traceability tab. 2. The User selects a scenario. 3. The system lists each step with the syntax nodes it was derived from. 4. The User selects a step; the system highlights the corresponding text in the source requirement.
Exit condition	The User has seen the requirement elements behind each step.
Quality requirements	Every generated step has at least one trace link.
Related requirements	FR-41, FR-42

Table 15. Use case UC-7 Export Gherkin

Use case	**UC-7 Export Gherkin**
Participating actors	User, File System
Entry condition	At least one GherkinFeature has been generated.
Flow of events	1. The User chooses Copy to clipboard, or Export .feature. 2. For export, the system shows the Export Dialog with a proposed file name. 3. The User confirms the folder and file name. 4. The system writes the .feature file(s) through the File System and confirms success.
Exit condition	The Gherkin text is on the clipboard or stored as .feature file(s).
Quality requirements	Exported files pass the Cucumber Gherkin parser (NFR-8).
Related requirements	FR-38, FR-39, FR-40

Table 16. Use case UC-8 View Error Report

Use case	**UC-8 View Error Report**
Participating actors	User
Entry condition	A processing stage has reported an error during UC-3 or UC-4.
Flow of events	1. The system stops processing the requirement. 2. The system displays the Error Report Panel with the error class (input or generation error), the stage, the problematic input, the reason and a suggested correction. 3. The User chooses Edit requirement (return to UC-1) or Dismiss.
Exit condition	The User knows why the requirement could not be processed; the requirement text is preserved.
Quality requirements	Plain-language messages (NFR-2).
Related requirements	FR-18, FR-43, FR-44, FR-45

Table 17. Use case UC-9 Manage Generation History

Use case	**UC-9 Manage Generation History**
Participating actors	User
Entry condition	History storage is enabled.
Flow of events	1. The User opens the History view. 2. The system lists stored GenerationRecords, newest first. 3. The User searches or filters the list. 4. The User reloads a record (the system restores the requirement, options and feature) or deletes it after confirmation.
Exit condition	The selected record is reloaded or deleted.
Quality requirements	History list opens within 1 s for 1,000 records.
Related requirements	FR-48, FR-49, FR-50

Table 18. Use case UC-10 Configure Generation Options

Use case	**UC-10 Configure Generation Options**
Participating actors	User
Entry condition	The Requirement Editor is open.
Flow of events	1. The User opens the Generation Options dialog. 2. The system shows the alternative-flow types with their current state. 3. The User enables or disables flow types and saves. 4. The system applies the options to subsequent generations, including all requirements of an imported collection.
Exit condition	The new options are active.
Quality requirements	Options persist for the session.
Related requirements	FR-31, FR-51, FR-52

3.4.3 Object model

The object model is kept at the conceptual level: classes show attributes and associations only, without operations or database elements. Following [3], the analysis objects are classified into entity, boundary and control objects. Class names are used consistently in the requirements, use cases and diagrams.

Table 19. Analysis objects by stereotype

Stereotype	Analysis objects
«entity»	Requirement, EarsRequirement, UserStory, RequirementSet, RequirementFormat, RequirementModel, SyntaxNode, RequirementNode, ActorNode, ActionNode, ConditionNode, TriggerNode, ExpectedBehaviorNode, OutcomeNode, GherkinFeature, GherkinScenario, GherkinStep, ProcessingError, InputError, GenerationError, FeatureFile, GenerationRecord
«boundary»	RequirementEditorForm, ScenarioView, TraceabilityView, ErrorReportPanel, ImportDialog, ExportDialog, CollectionSummaryView, HistoryView, GenerationOptionsDialog
«control»	ProcessingPipeline, ProcessingStage (SyntaxValidationStage, TokenizationStage, StructuralValidationStage, SyntaxTreeConstructionStage, ScenarioConstructionStage), FormatAdapter (EarsRequirementAdapter, UserStoryAdapter), ScenarioBuilder, GherkinFormatter, HistoryManager

Requirement and syntax tree model

Figure 2 shows the requirement side of the domain. Requirement is abstract and specialized into EarsRequirement and UserStory, which keep their format-specific structure. Each requirement can be adapted into exactly one RequirementModel, the format-independent representation used by the rest of the system. The RequirementModel owns the root of the syntax tree. The syntax tree follows a composite structure: every node is a SyntaxNode, and a RequirementNode is composed of the other nodes. A RequirementNode has exactly one

ActorNode, exactly one ActionNode, one or more ExpectedBehaviorNodes, zero or one TriggerNode, zero or one OutcomeNode and zero or more ConditionNodes; these multiplicities are the rules enforced by the structural validation stage.

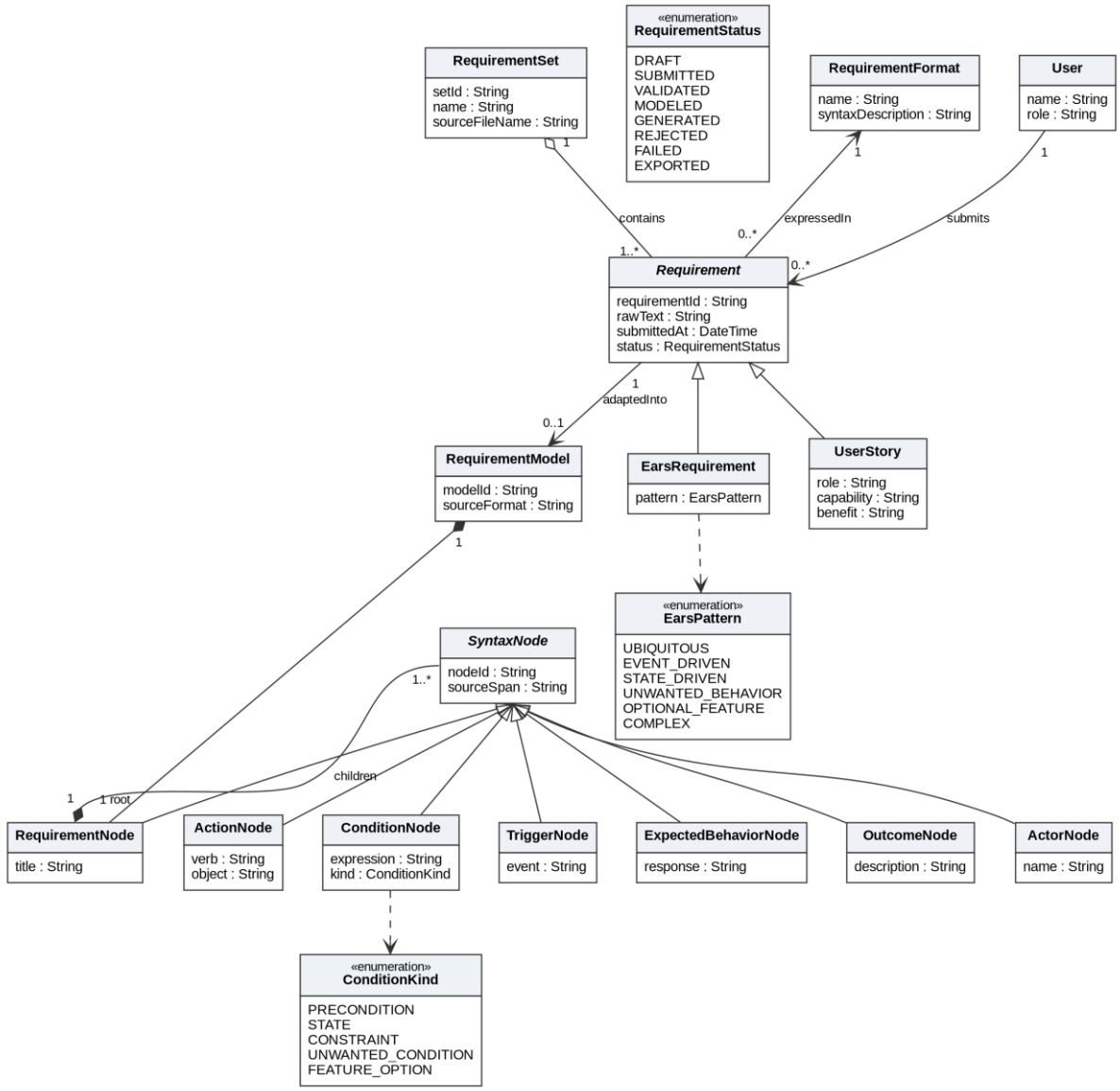


Figure 2. Class diagram: requirement and syntax tree model

Processing, generation and output model

Figure 3 shows the processing and output side. ProcessingPipeline is an ordered composition of ProcessingStages; the reflexive **next** association expresses the Chain of Responsibility, where each stage either passes the requirement to its successor or reports a ProcessingError. ProcessingError is abstract and distinguishes InputError (defects in the requirement) from GenerationError (failures of the system), which satisfies FR-44. FormatAdapter is the Adapter abstraction: EarsRequirementAdapter and UserStoryAdapter translate their format-specific requirements into a RequirementModel, so that scenario construction never depends on the original format. ScenarioBuilder is the Builder concept that constructs GherkinScenario objects; a GherkinScenario owns at least two GherkinSteps (one When and one Then) and each GherkinStep traces to the syntax nodes it was derived from. GenerationRecord represents the optional history entry.

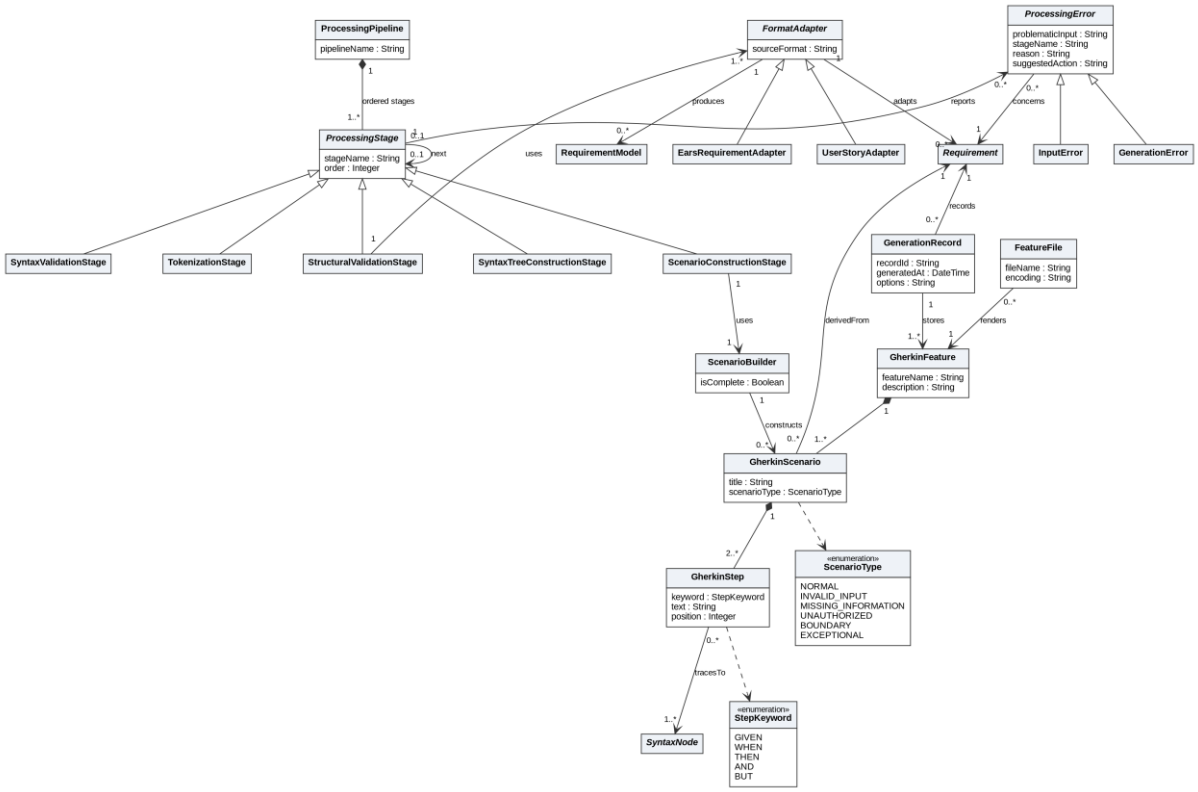


Figure 3. Class diagram: processing, generation and output model

Class dictionary

Table 20. Class dictionary

Class	Description
Requirement	Abstract statement of required behavior submitted by a User; has an identifier, raw text and a status.
EarsRequirement / UserStory	Format-specific requirements: an EARS sentence with its pattern; a user story with role, capability and benefit.
RequirementSet	A collection of requirements imported from one file.
RequirementFormat	A supported input format (EARS, User Story) and its syntax description.
RequirementModel	Unified, format-independent representation of a requirement; owns the syntax tree.
SyntaxNode and subclasses	Typed logical elements of a requirement (actor, action, condition, trigger, expected behavior, outcome) with the text span they come from.
ProcessingPipeline / ProcessingStage	The ordered chain of processing steps; each stage handles one concern and passes the requirement on or reports an error.
FormatAdapter	Converts a format-specific requirement into a RequirementModel.
ScenarioBuilder	Assembles one GherkinScenario step by step and refuses incomplete scenarios.
GherkinFeature / GherkinScenario / GherkinStep	The generated BDD artifacts: a feature groups scenarios; a scenario has a title, a type and ordered steps; a step has a keyword and text.
ProcessingError / InputError / GenerationError	Structured error with problematic input, stage, reason and suggested correction.
FeatureFile	A .feature file rendering of a GherkinFeature.
GenerationRecord	Stored history entry linking a requirement to its generated features and options.
GherkinFormatter, HistoryManager	Control objects that turn features into Gherkin text and manage stored records.

3.4.4 Dynamic model

The dynamic model describes how the analysis objects collaborate to realize the main use cases and how the most important objects change state.

Sequence diagram: UC-4 Generate BDD Scenarios (successful flow)

Figure 4 shows the message flow when a valid requirement is generated. The requirement is handed from stage to stage along the chain; the structural validation stage delegates format conversion to the FormatAdapter, and the scenario construction stage uses the ScenarioBuilder once per identified flow.

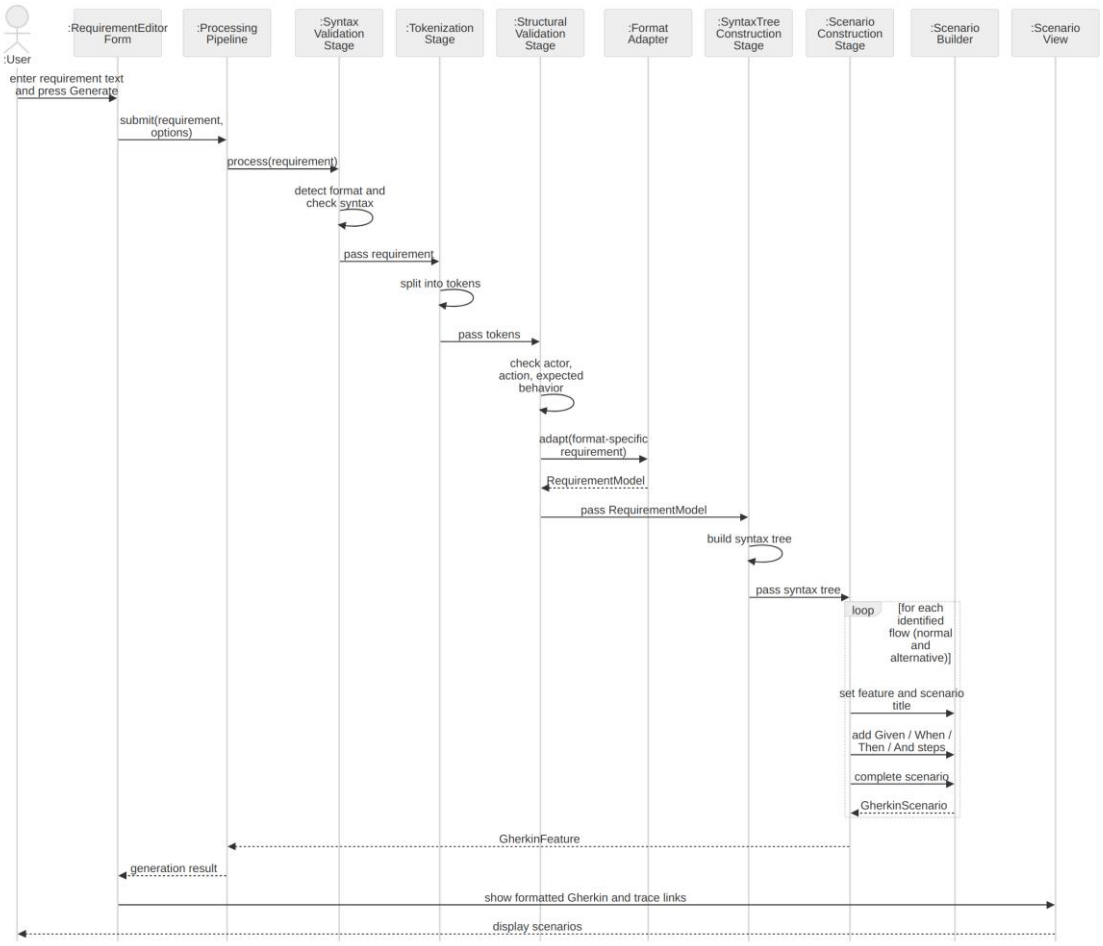


Figure 4. Sequence diagram for UC-4 Generate BDD Scenarios

Sequence diagram: UC-8 View Error Report (incomplete requirement)

Figure 5 corresponds to scenario incompleteRequirementFeedback. The structural validation stage detects the missing expected behavior, stops the chain and returns an InputError; the remaining stages are never invoked.

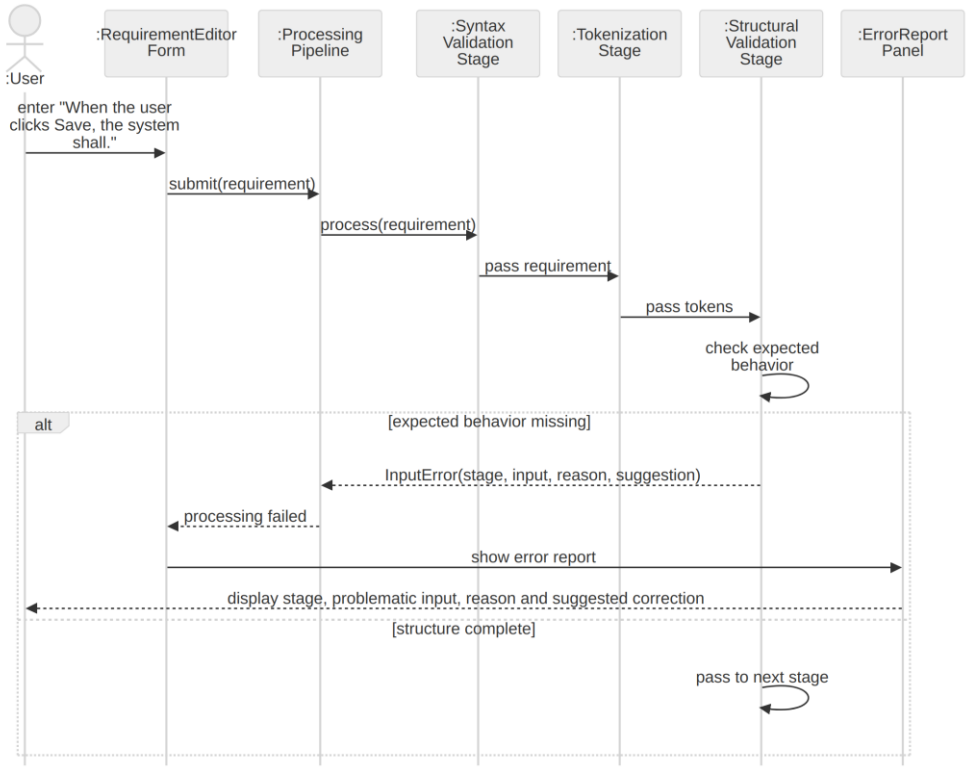


Figure 5. Sequence diagram for UC-8 View Error Report

Activity diagram: requirement processing

Figure 6 summarizes the control flow of the processing pipeline, including every decision that leads to an error report.

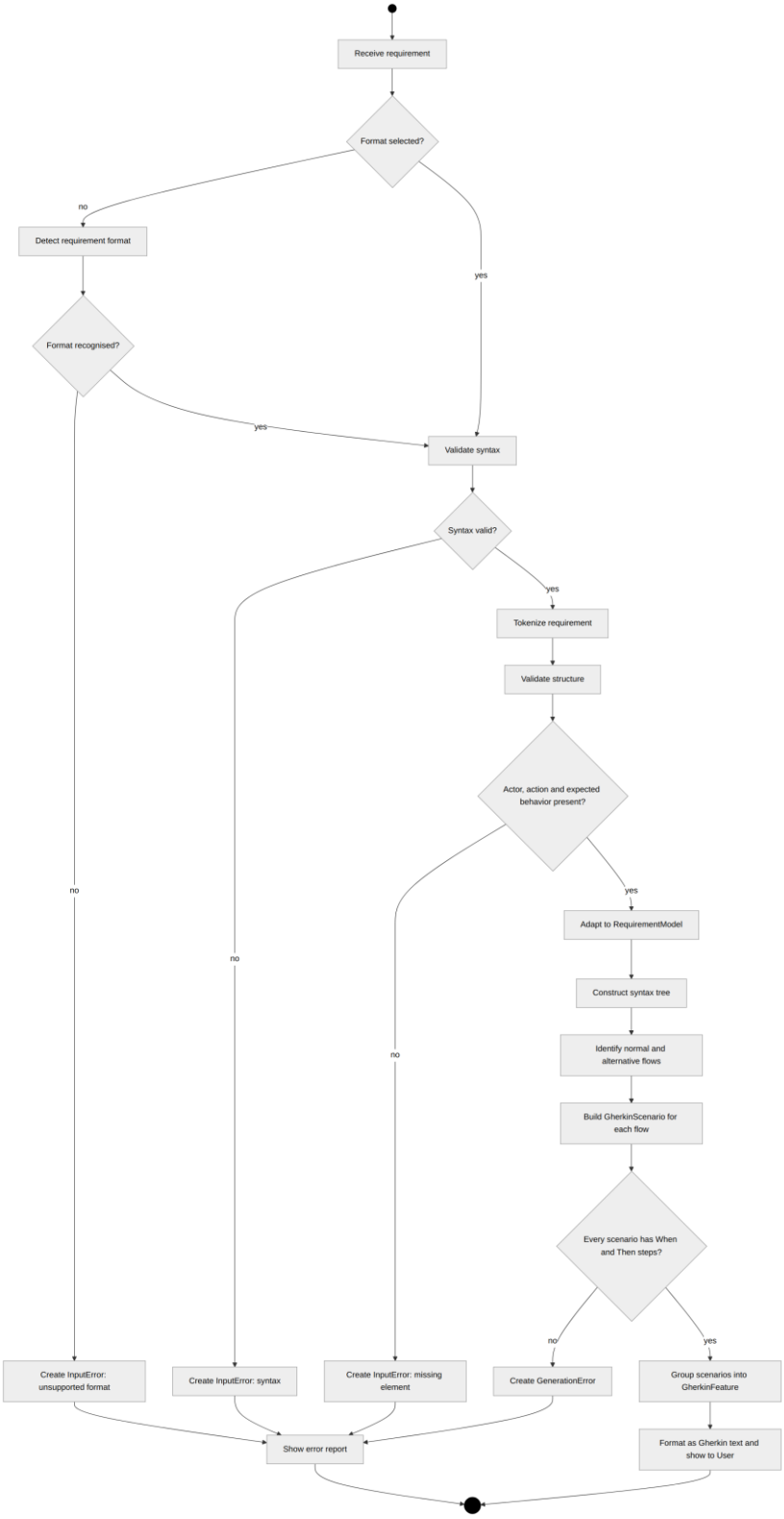


Figure 6. Activity diagram of requirement processing

State machine diagram: Requirement

Figure 7 shows the life cycle of a Requirement, which corresponds to the RequirementStatus enumeration. A rejected or failed requirement returns to Draft when the User edits it.

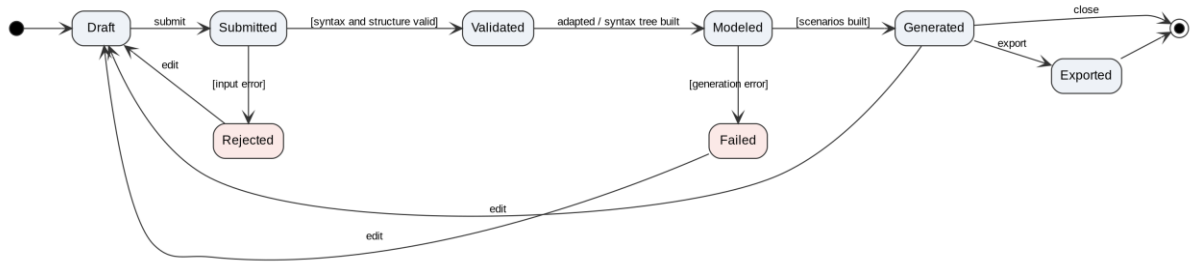


Figure 7. State machine diagram of Requirement

State machine diagram: ScenarioBuilder

Figure 8 shows the states of a ScenarioBuilder while one scenario is constructed. Completing the scenario is only possible in the Complete state, which requires at least one When step and one Then step (FR-34); in all other states the attempt leads to a GenerationError.

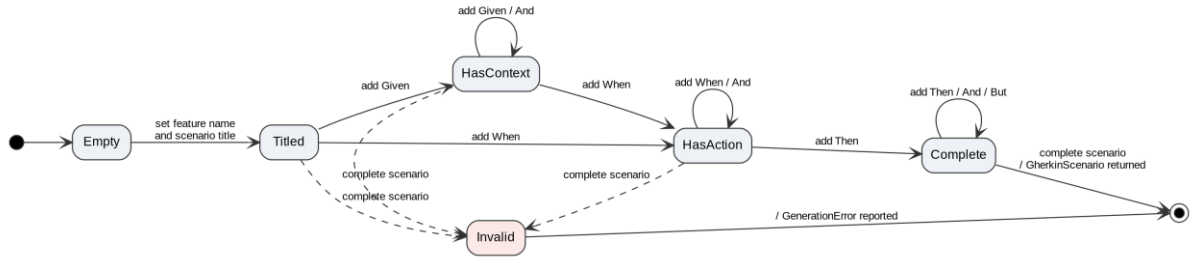


Figure 8. State machine diagram of ScenarioBuilder

3.4.5 User interfaces and Mock-ups

The mock-ups below are low-fidelity wireframes of the main screens. The Requirement Editor and the Scenario View share one main window: the left pane is used for input and the right pane for output, so the User always sees the requirement and the generated Gherkin side by side.

Navigational paths

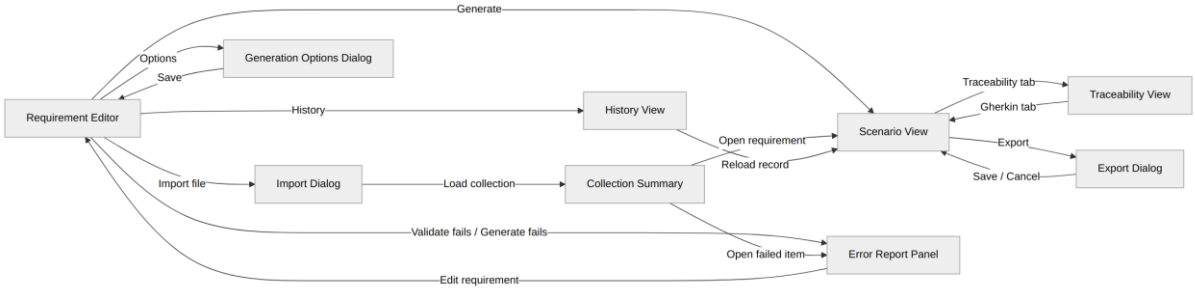


Figure 9. Navigation between screens

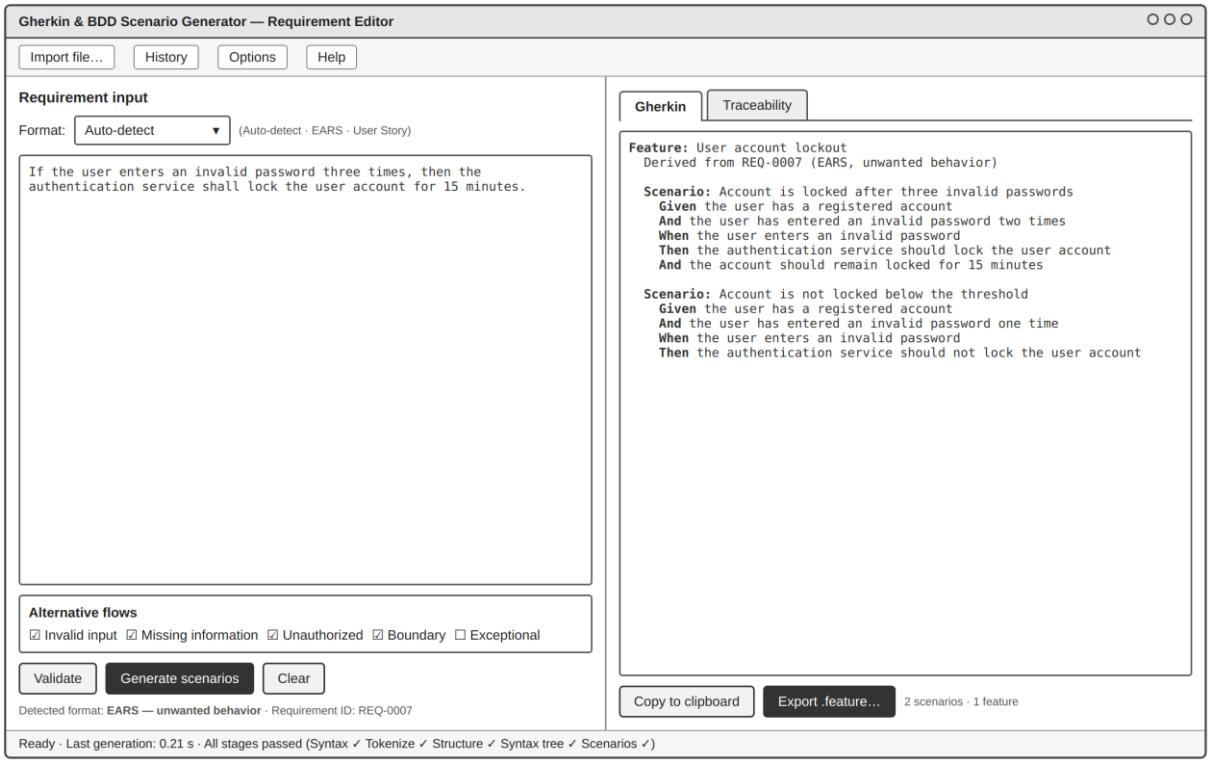


Figure 10. Mock-up: Requirement Editor with generated Gherkin (UC-1, UC-4, UC-5, UC-7)

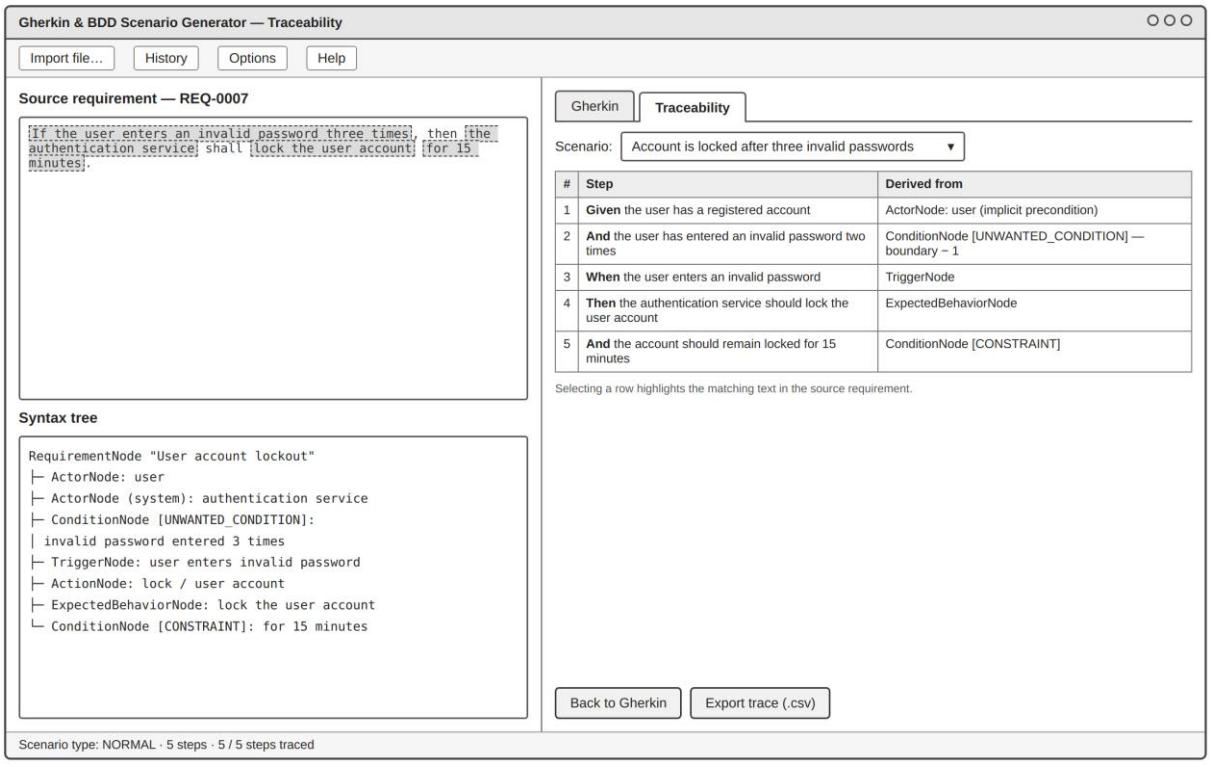


Figure 11. Mock-up: Traceability view (UC-6)

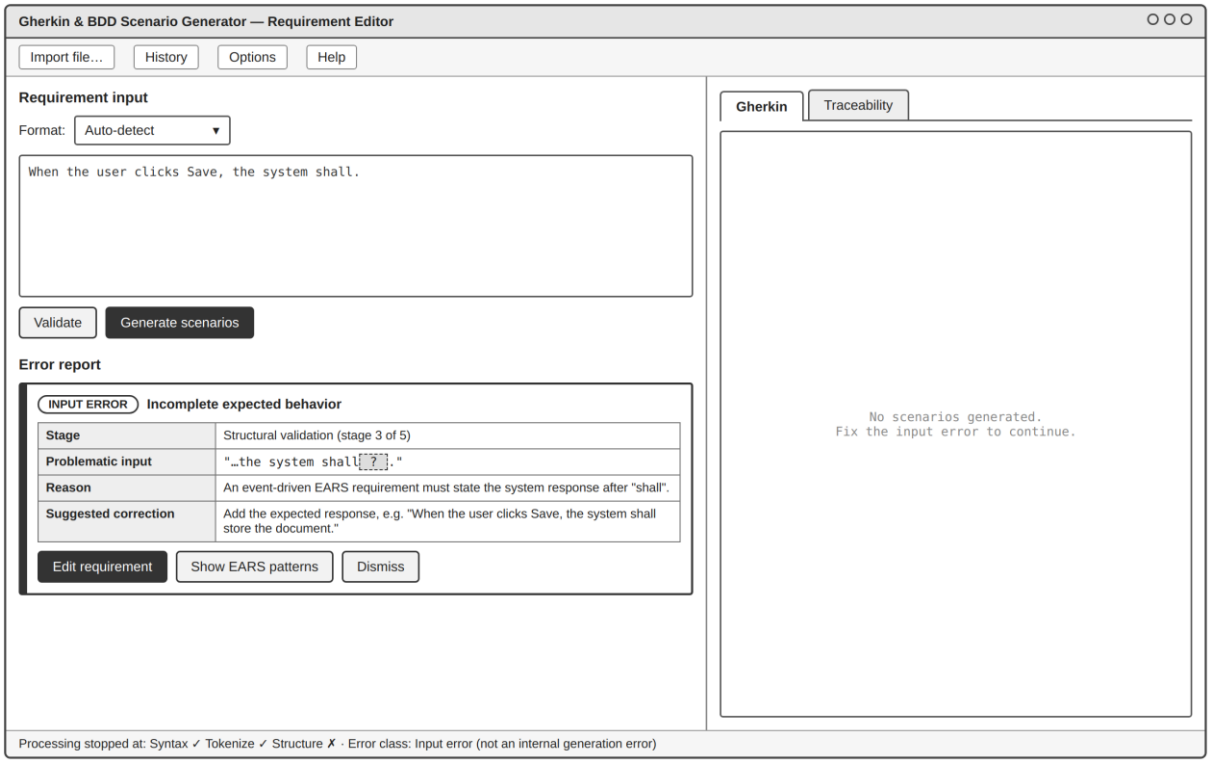


Figure 12. Mock-up: Error report for an incomplete requirement (UC-3, UC-8)

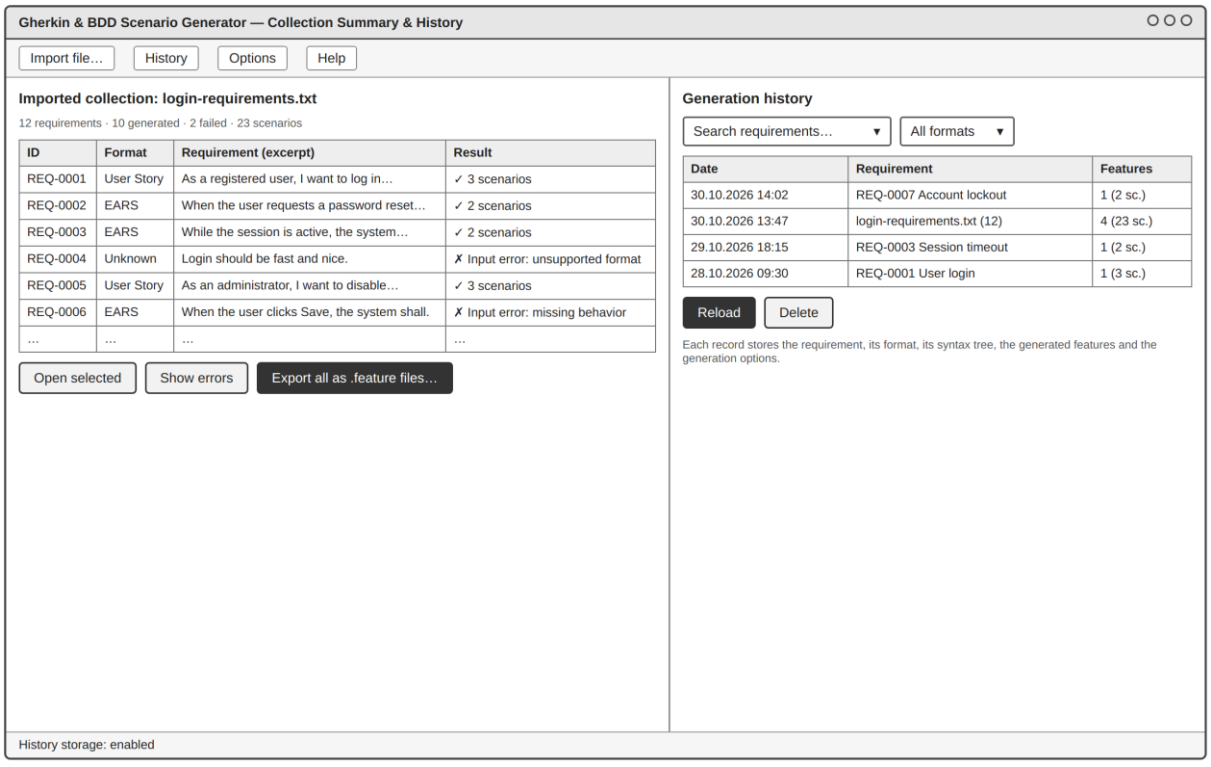


Figure 13. Mock-up: Collection summary and generation history (UC-2, UC-9)

4 Glossary of Terms

The glossary defines the domain terms used in this document. The same terms are used as class names in the object model.

Term	Definition
Actor (requirement)	The person, role or system that performs or initiates the behavior described by a requirement; represented by an ActorNode.
Alternative flow	A scenario that describes behavior other than the normal successful path: invalid input, missing information, unauthorized operation, boundary condition or exceptional behavior.
Behavior-Driven Development (BDD)	A development practice in which the expected behavior of software is specified as examples in a shared language before implementation.
Boundary condition	A situation at or next to a limit stated in a requirement, for example the third failed login attempt.
EARS	Easy Approach to Requirements Syntax [4]; a set of six sentence patterns (ubiquitous, event-driven, state-driven, unwanted behavior, optional feature, complex) for writing requirements.
EARS pattern	One of the six EARS sentence templates, identified by the keywords When, While, If…then, Where or their absence.
Expected behavior	The response the system shall exhibit, normally the phrase after "shall"; represented by an ExpectedBehaviorNode.
Feature (Gherkin)	The top-level element of a Gherkin file that groups related scenarios; represented by a GherkinFeature.
Feature file	A text file with the .feature extension that contains one Gherkin feature.
Generation error	An error caused by a failure of the system while constructing scenarios, as opposed to a defect in the input requirement.
Generation options	The User's choice of which alternative-flow types are generated.
GenerationRecord	A stored history entry that links a requirement with its format, syntax tree, generated features and options.
Gherkin	The structured language used by BDD tools such as Cucumber, based on the keywords Feature, Scenario, Given, When, Then, And and But [6].
Given / When / Then step	The three main step types of a scenario: context (Given), event or action (When) and expected outcome (Then). And and But continue the previous step type.
Input error	An error caused by a missing, invalid or unsupported element in the requirement provided by the User.
Processing stage	One link in the processing chain (syntax validation, tokenization, structural validation, syntax tree construction, scenario construction).
Requirement	A statement of required system behavior submitted to the system, written as an EARS sentence or a user story.
Requirement collection	A set of requirements imported together from one file; represented by a RequirementSet.
RequirementModel	The unified, format-independent internal representation of a requirement.
Scenario	A concrete example of system behavior consisting of a title and ordered Given/When/Then steps; represented by a GherkinScenario.
Syntax tree	The tree of typed SyntaxNodes that represents the logical structure of a requirement (the requirement AST).
Token	A word or symbol produced when the requirement text is split during tokenization.
Traceability (trace link)	The recorded relationship between a generated step and the requirement elements it was derived from.
User	Any human actor of the system: Developer, Test Engineer or Business Analyst.
User story	A requirement of the form "As a <role>, I want <capability>, so that <benefit>" [7].

5 Priority

Requirements are prioritized with the MoSCoW scheme. **Must** requirements form the minimum system that demonstrates the full transformation pipeline and the three required design patterns; they are implemented first. **Should** requirements add important usability and coverage and are planned for the same release if time allows. **Could** requirements (history storage and boundary-value generation) are desirable and will be implemented after all Must and Should requirements. No requirement in this document is classified as Won't; the out-of-scope items of Section 1.2 are the Won't items of this release.

Table 21. Priority of functional requirements

Priority	Functional requirements	Rationale
Must	FR-1, FR-3, FR-4, FR-6, FR-7, FR-8, FR-9, FR-10, FR-11, FR-12, FR-13, FR-14, FR-15, FR-16, FR-17, FR-18, FR-19, FR-20, FR-21, FR-22, FR-25, FR-26, FR-27, FR-28, FR-29, FR-33, FR-34, FR-35, FR-36, FR-37, FR-39, FR-41, FR-43, FR-44	Required for an end-to-end transformation from requirement to valid Gherkin with meaningful errors; directly demanded by the problem statement.
Should	FR-2, FR-5, FR-23, FR-24, FR-30, FR-31, FR-38, FR-40, FR-42, FR-45, FR-46, FR-47, FR-51, FR-52	Improve coverage, usability and batch processing; the core pipeline works without them.
Could	FR-32, FR-48, FR-49, FR-50	Optional persistence and additional boundary scenarios; the problem statement marks the database as optional.

Table 22. Priority of nonfunctional requirements

Priority	Nonfunctional requirements
Must	NFR-2, NFR-6, NFR-7, NFR-8, NFR-9, NFR-13, NFR-14, NFR-18, NFR-19, NFR-20, NFR-21, NFR-24, NFR-26, NFR-27, NFR-28, NFR-31, NFR-32
Should	NFR-1, NFR-3, NFR-10, NFR-11, NFR-15, NFR-16, NFR-17, NFR-23, NFR-25, NFR-29, NFR-30
Could	NFR-4, NFR-5, NFR-12, NFR-22

6 Traceability

Traceability is documented at three levels: from the sections of the project problem statement [1] to the requirements, from the use cases to the functional requirements, and from every functional requirement to its priority, use cases and analysis classes.

Problem statement to requirements

Table 23. Traceability from the problem statement to requirements

Problem statement section	Requirements	Main analysis classes
Requirement Input	FR-1 – FR-8	Requirement, RequirementSet, EarsRequirement, UserStory
Requirement Parsing	FR-9 – FR-12	TokenizationStage, StructuralValidationStage
Requirement Abstract Syntax Tree	FR-14, FR-15	RequirementNode, SyntaxNode subclasses
Requirement Validation (Chain of Responsibility)	FR-16 – FR-24, NFR-14, NFR-19	ProcessingPipeline, ProcessingStage
Input Adaptation (Adapter)	FR-13, NFR-13, NFR-19	FormatAdapter, EarsRequirementAdapter, UserStoryAdapter, RequirementModel
BDD Scenario Generation	FR-25 – FR-29, FR-33, FR-35	ScenarioConstructionStage, GherkinFeature
Gherkin Scenario Construction (Builder)	FR-26 – FR-29, FR-34, NFR-19	ScenarioBuilder, GherkinScenario, GherkinStep
Scenario Formatting and Output	FR-36 – FR-42, NFR-15, NFR-24	GherkinFormatter, FeatureFile, TraceabilityView
Multiple Scenarios and Alternative Flows	FR-30 – FR-32, FR-51	ConditionNode, ScenarioType
Error Handling and User Feedback	FR-18 – FR-23, FR-43 – FR-47, NFR-2	ProcessingError, InputError, GenerationError, ErrorReportPanel
Extensibility	NFR-13 – NFR-15	FormatAdapter, ProcessingStage
Technical Architecture and Database	FR-48 – FR-50, NFR-20, NFR-22	GenerationRecord, HistoryManager

Use cases to functional requirements

Table 24. Traceability from use cases to functional requirements

Use case	Functional requirements
UC-1 Enter Requirement	FR-1, FR-3, FR-4, FR-5, FR-6, FR-7, FR-8
UC-2 Import Requirement Collection	FR-2, FR-7, FR-46, FR-47, FR-52
UC-3 Validate Requirement	FR-9, FR-10, FR-11, FR-12, FR-16, FR-17, FR-18, FR-19, FR-20, FR-21, FR-22, FR-23, FR-24
UC-4 Generate BDD Scenarios	FR-13, FR-14, FR-15, FR-16, FR-17, FR-25, FR-26, FR-27, FR-28, FR-29, FR-30, FR-31, FR-32, FR-33, FR-34, FR-35, FR-41
UC-5 View Generated Scenarios	FR-36, FR-37
UC-6 Review Traceability	FR-41, FR-42
UC-7 Export Gherkin	FR-38, FR-39, FR-40
UC-8 View Error Report	FR-18, FR-43, FR-44, FR-45
UC-9 Manage Generation History	FR-48, FR-49, FR-50
UC-10 Configure Generation Options	FR-31, FR-51, FR-52

Functional requirements to use cases and classes

Table 25. Traceability matrix of functional requirements

FR	Priority	Use cases	Analysis classes
FR-1	Must	UC-1	Requirement, RequirementEditorForm
FR-2	Should	UC-2	RequirementSet, ImportDialog
FR-3	Must	UC-1	EarsRequirement, EarsPattern
FR-4	Must	UC-1	UserStory
FR-5	Should	UC-1	RequirementFormat, SyntaxValidationStage
FR-6	Must	UC-1	RequirementFormat, SyntaxValidationStage
FR-7	Must	UC-1, UC-2	Requirement, SyntaxValidationStage
FR-8	Must	UC-1	Requirement, SyntaxValidationStage
FR-9	Must	UC-3	TokenizationStage
FR-10	Must	UC-3	EarsRequirement, EarsPattern, TokenizationStage
FR-11	Must	UC-3	UserStory, TokenizationStage
FR-12	Must	UC-3	StructuralValidationStage, SyntaxNode
FR-13	Must	UC-4	FormatAdapter, EarsRequirementAdapter, UserStoryAdapter, RequirementModel
FR-14	Must	UC-4	SyntaxTreeConstructionStage, RequirementNode
FR-15	Must	UC-4	SyntaxNode and its subclasses
FR-16	Must	UC-3, UC-4	ProcessingPipeline, ProcessingStage
FR-17	Must	UC-3, UC-4	ProcessingPipeline, ProcessingStage
FR-18	Must	UC-3, UC-8	ProcessingStage, ProcessingError
FR-19	Must	UC-3	SyntaxValidationStage, InputError
FR-20	Must	UC-3	StructuralValidationStage, InputError
FR-21	Must	UC-3	StructuralValidationStage, InputError
FR-22	Must	UC-3	StructuralValidationStage, InputError
FR-23	Should	UC-3	StructuralValidationStage, InputError
FR-24	Should	UC-3	ProcessingPipeline, RequirementEditorForm
FR-25	Must	UC-4	ScenarioConstructionStage, ScenarioBuilder, GherkinScenario, GherkinStep
FR-26	Must	UC-4	ScenarioConstructionStage, ScenarioBuilder, GherkinScenario, GherkinStep
FR-27	Must	UC-4	ScenarioConstructionStage, ScenarioBuilder, GherkinScenario, GherkinStep
FR-28	Must	UC-4	ScenarioConstructionStage, ScenarioBuilder, GherkinScenario, GherkinStep
FR-29	Must	UC-4	ScenarioConstructionStage, ScenarioBuilder, GherkinScenario, GherkinStep
FR-30	Should	UC-4	ScenarioConstructionStage, ConditionNode, ScenarioType
FR-31	Should	UC-4, UC-10	ScenarioConstructionStage, ConditionNode, ScenarioType
FR-32	Could	UC-4	ScenarioConstructionStage, ConditionNode, ScenarioType
FR-33	Must	UC-4	ScenarioConstructionStage, ScenarioBuilder, GherkinScenario, GherkinStep
FR-34	Must	UC-4	ScenarioBuilder, GenerationError
FR-35	Must	UC-4	GherkinFeature
FR-36	Must	UC-5	GherkinFormatter, GherkinFeature
FR-37	Must	UC-5	ScenarioView
FR-38	Should	UC-7	ExportDialog, FeatureFile
FR-39	Must	UC-7	ExportDialog, FeatureFile
FR-40	Should	UC-7	ExportDialog, FeatureFile
FR-41	Must	UC-4, UC-6	GherkinStep, SyntaxNode
FR-42	Should	UC-6	TraceabilityView
FR-43	Must	UC-8	ErrorReportPanel, ProcessingError
FR-44	Must	UC-8	InputError, GenerationError
FR-45	Should	UC-8	InputError, GenerationError
FR-46	Should	UC-2	ProcessingPipeline, CollectionSummaryView
FR-47	Should	UC-2	ProcessingPipeline, CollectionSummaryView
FR-48	Could	UC-9	GenerationRecord, HistoryManager, HistoryView
FR-49	Could	UC-9	GenerationRecord, HistoryManager, HistoryView

FR	Priority	Use cases	Analysis classes
FR-50	Could	UC-9	GenerationRecord, HistoryManager, HistoryView
FR-51	Should	UC-10	GenerationOptionsDialog, GenerationRecord
FR-52	Should	UC-2, UC-10	GenerationOptionsDialog, GenerationRecord